

06026240 INTELLIGENT SYSTEM DEVELOPMENT

Chapter III

Deep learning (cont.)

with Dr. Bhattarabhorn Wattanacheep

How to build a CNNs



What is TensorFlow?

developed by Google Brain in 2015

TensorFlow shines in production environments. It's trusted by many companies for its stability and reliability.

Community Support : You're in good company with a vast and active community that can help you troubleshoot issues.

User-Friendly : TensorFlow 2.0+ has improved its user-friendliness, making it easier to work with.



What is PyTorch?

Developed by Facebook's AI Research lab (FAIR),

“An open source machine learning framework that accelerates the path from research prototyping to production deployment.”

K

What is Keras?

Initially developed as a stand-alone high-level neural network API, Keras is now a part of TensorFlow.

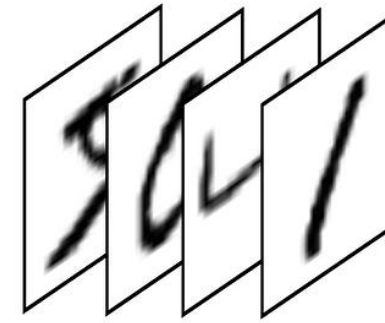
Simplicity: Keras excels in its simplicity, making it ideal for beginners and rapid prototyping.

Ease of Use: You can switch between CPU and GPU without breaking a sweat.



scikit-learn

เป็นเครื่องมือในการทำ machine learning อีกหนึ่งตัวที่น่าศึกษาเอาไว้ เนื่องจากความง่ายและมีประสิทธิภาพในการทำ predictive data analysis โดยสร้างขึ้นมาจากโมดูลหลายตัวทั้ง NumPy, SciPy และ matplotlib ส่งผลให้ scikit-learn มีความสามารถที่หลากหลาย นำไปใช้ในงานได้หลายประเภท



MNIST

- Famous introductory computer vision entry dataset, beginner dataset, starter or benchmark dataset
- each row is an image
- label column ranges from 0 to 9. There are ten classes
- Each image is 28x28 pixels
- Dimension sometimes written as 1x28x28 or 28x28x1, the extra dimension is the number of color channels. One means gray scale. Three is normal RGB three color channel.
- Each pixel value ranges from 0 to 255
- it's common to flatten the image into a vector of length 784
- in grayscale, a black pixel is encoded as 0, a white pixel is encoded as 255

MNIST

There are 70,000 images. Each MNIST image is a handwritten digit between 0 and 9 written by public employees and census bureau workers and high school students.

MNIST dataset is considered “solved”. Most modern ML and DL architecture can easily achieve above 95%+ result with minimal training. In the past there were some challenges: 1 looks like 7, 3 looks like 8, 4 looks like 9 in hand writing.

Fashion MNIST

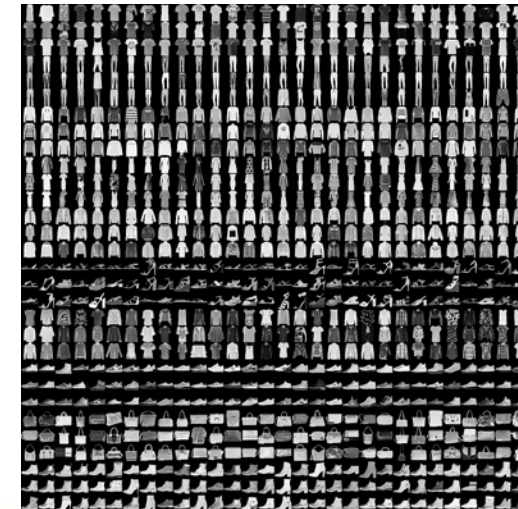
A more sophisticated alternative image dataset to MNIST handwritten digits, consisted of images of clothing such as pants, t-shirts etc.

Contains 70,000 grayscale images in 10 categories, 28x28 pixel each (just like MNIST). Great starter dataset for getting started or learning Convolutional Neural Networks. In other words, Fashion MNIST is a great Hello World dataset for trying out Convolutional Neural Networks (CNN).

The nice 70,000 image volume generously allow us to use 60,000 images for training, 10,000 for evaluation.

labels:

- | | |
|---------------|--------------|
| 0 T-shirt/top | 5 Sandal |
| 1 Trouser | 6 Shirt |
| 2 Pullover | 7 Sneaker |
| 3 Dress | 8 Bag |
| 4 Coat | 9 Ankle boot |



CIFAR

CIFAR pronounced C-FAR (see-far). CIFAR10, CIFAR100, the number suffix refers to the number of classes/categories.

“Dataset of 50,000 32x32 color training images, labeled over 10 categories, and 10,000 test images.” “Dataset of 50,000 32x32 color training images, labeled over 100 categories, and 10,000 test images.” — Keras Dataset

The CIFAR10 dataset. It has the classes: ‘airplane’, ‘automobile’, ‘bird’, ‘cat’, ‘deer’, ‘dog’, ‘frog’, ‘horse’, ‘ship’, ‘truck’. The images in CIFAR-10 are of size 3x32x32, i.e. 3-channel color images of 32x32 pixels in size

Famous Machine Learning Datasets



KMITL
สถาบันเทคโนโลยี
พระจอมเกล้า
เจ้าพระยา

KMITL
FIGHT

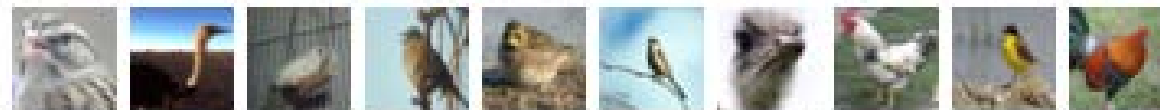
airplane



automobile



bird



cat



deer



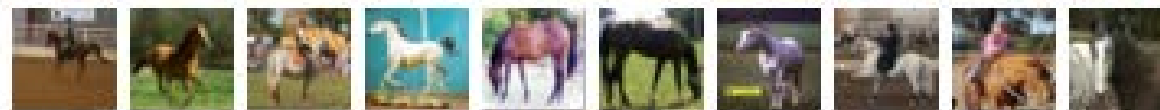
dog



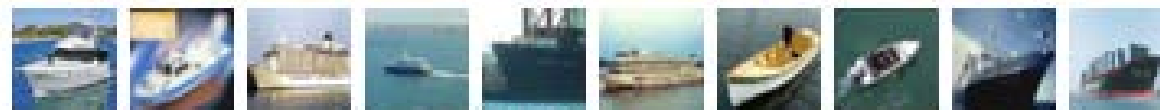
frog



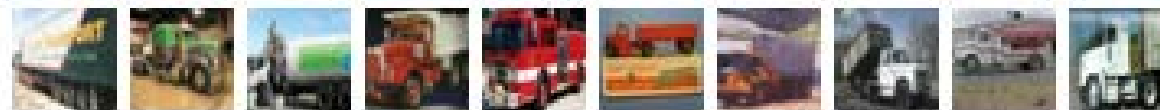
horse



ship



truck



Titanic Survivor Dataset

The perfect entry, beginner friendly, playground introduction dataset to compete on Kaggle. It is a binary classification task predicts 1, 0 whether a passenger survived or not.

Street View House Number (SVHN) Dataset

Contains Google Map Street View house number images, SVHN is amore sophisticated, non-trivial, colorful alternative to MNIST.

Boston Housing Dataset

Predict medium housing price in Boston, great for regression models.

ImageNet

State of art largest image datasets and host of data competition, in which the top researchers compete. Measure top-5-error rate.

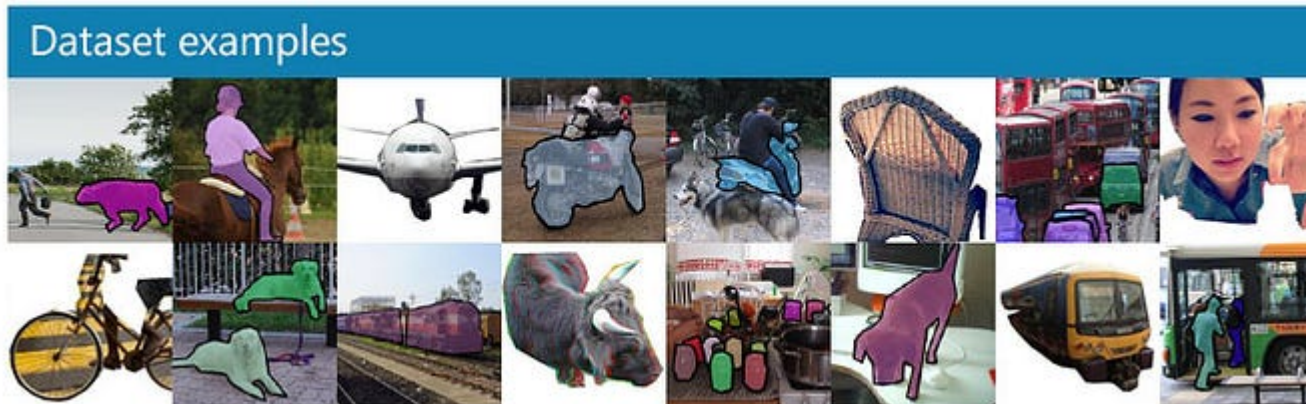
Image Recognition



ImageNet can contain 20,000 classes but the most frequently referenced ImageNet competition usually uses 1000 classes.

COCO Dataset

You can search and view the COCO dataset here. <http://cocodataset.org>



“COCO is a large-scale object detection, segmentation, and captioning dataset. COCO has several features: Object segmentation Recognition in context Superpixel stuff segmentation 330K images (>200K labeled) 1.5 million object instances 80 object categories 91 stuff categories

Step

1

Define the
Problem and
Set
Objectives

2

Prepare the
Data

3

Train the
Model

4

Evaluate on
the Test Set

5

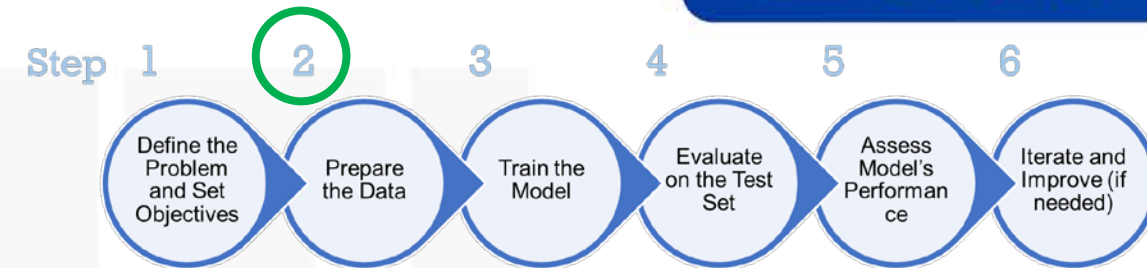
Assess
Model's
Performance

6

Iterate and
Improve (if
needed)

import libraries

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.callbacks import ReduceLROnPlateau, EarlyStopping, ModelCheckpoint
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
import os
import keras_tuner
import pandas as pd
import seaborn as sns
```



Classification

รูปแบบ dataset => file data อยู่บนเครื่อง

```
directory="C:/Users/NROSKOO/Documents/Python Scripts/folder/animal_data"
folder_names=["Bear","Bird","Cat","Cow","Deer","Dog","Dolphin","Elephant","Giraffe",
    "Horse","Kangaroo","Lion","Panda","Tiger","Zebra"]
print(folder_names,"\n",len(folder_names))
```

Step 1

2

3

4

5

6

Define the
Problem
and Set
Objectives

Prepare
the Data

Train the
Model

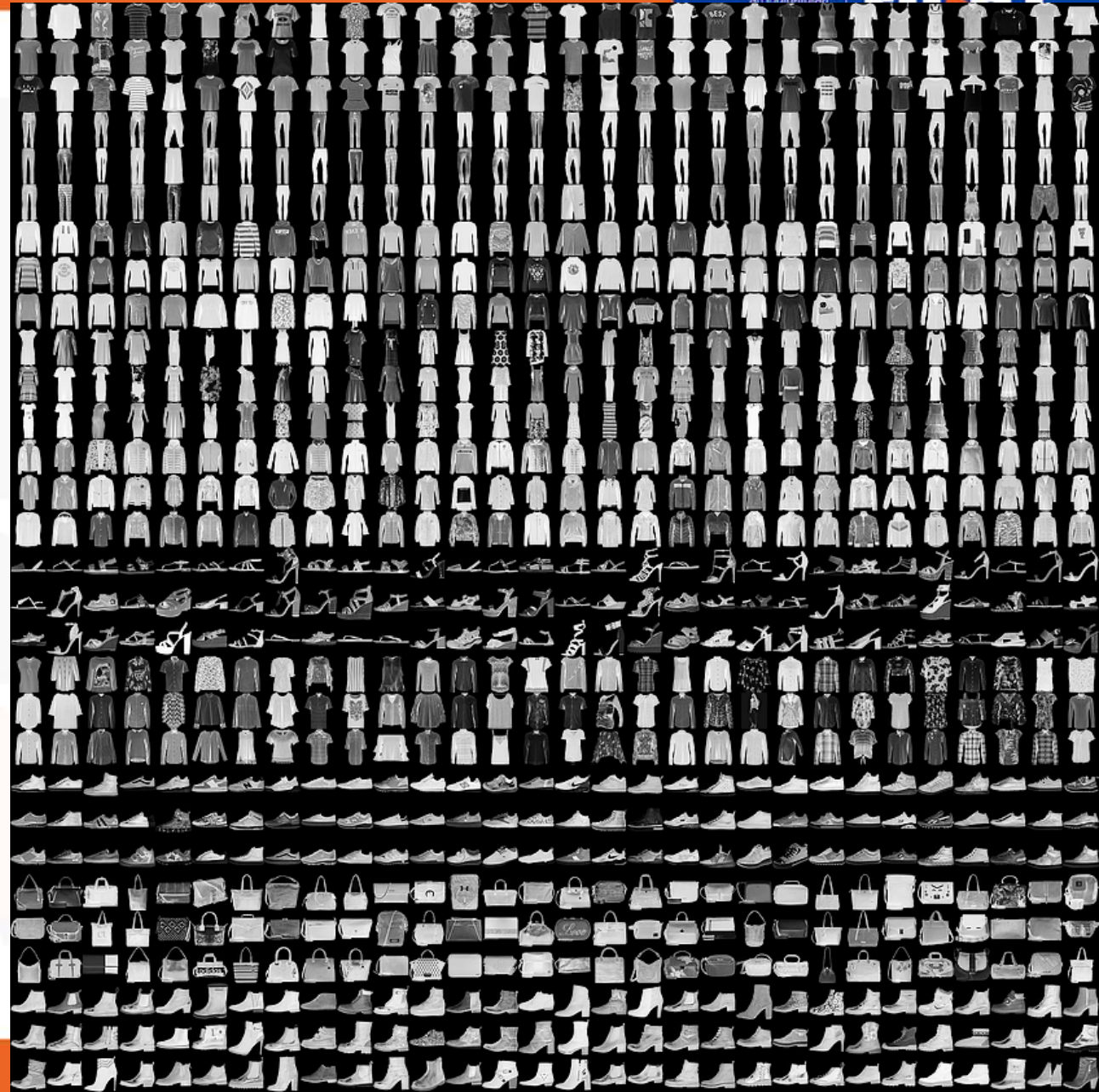
Evaluate
on the Test
Set

Assess
Model's
Performance

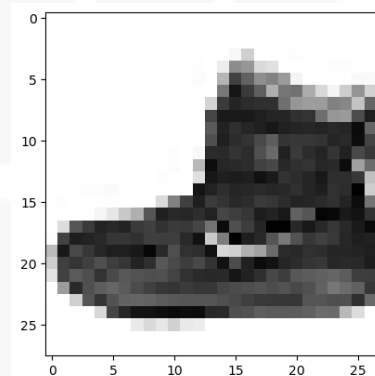
Iterate and
Improve (if
needed)

```
(X_train,y_train),(X_test, y_test) =  
keras.datasets.fashion_mnist.load_data()  
X_train.shape,y_train.shape  
X_test.shape,y_test.shape
```

```
class_labels = ['T-shirt/Top',  
'Trouser', 'Pullover', 'Dress',  
'coat', 'Sandal', 'shirt',  
'Sneakers', 'Bag', 'Ankle Boot']
```



```
plt.imshow(X_train[0], cmap='Greys')
```



Step 1

Define the
Problem and Set
Objectives

2

Prepare
the Data

3

Train the
Model

4

Evaluate
on the Test
Set

5

Assess
Model's
Performan
ce

6

Iterate and
Improve (if
needed)

Step 1

Define the
Problem
and Set
Objectives

2

Prepare
the Data

3

Train the
Model

4

Evaluate
on the Test
Set

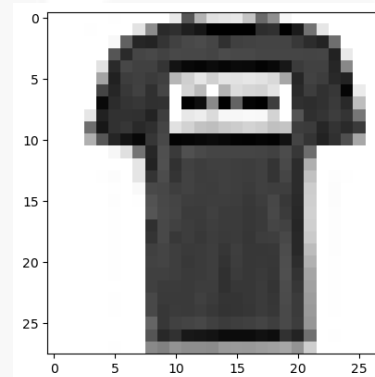
5

Assess
Model's
Performan
ce

6

Iterate and
Improve (if
needed)

```
plt.imshow(X_train[1], cmap='Greys')
```



Step 1

2

3

4

5

6



Define the Problem and Set Objectives

Prepare the Data

Train the Model

Evaluate on the Test Set

Assess Model's Performance

Iterate and Improve (if needed)

```
plt.figure(figsize =(16,16))
```

```
j=1
```

```
for i in np.random.randint(0,1000,25):
```

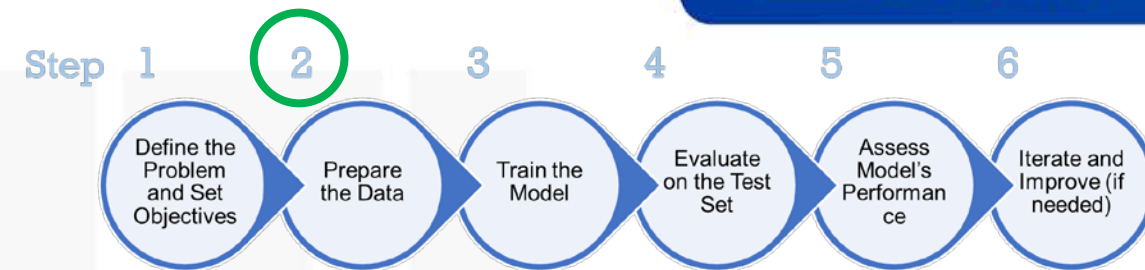
```
    plt.subplot(5,5,j);j+=1
```

```
    plt.imshow(X_train[i], cmap = 'Greys')
```

```
    plt.axis('off')
```

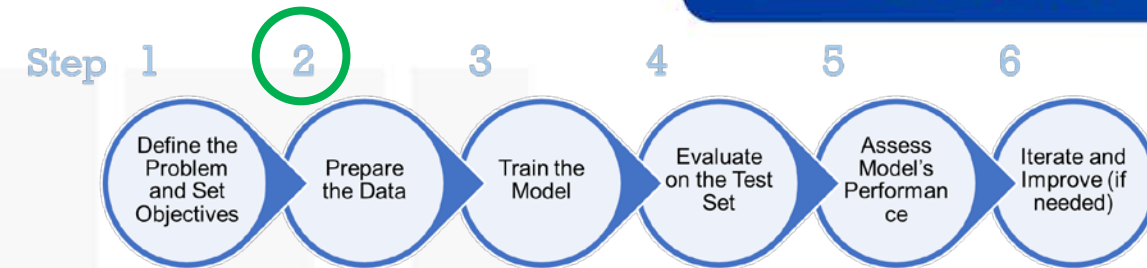
```
    plt.title('{} / {}'.format(class_labels[y_train[i]],  
                               y_train[1]))
```





```
print(X_train.shape)
X_train.ndim
```

```
# changing dimension by adding dim
X_train_add_dim = np.expand_dims(X_train,-1)
print(X_train_add_dim.shape)
X_test_add_dim = np.expand_dims(X_test,-1)
print(X_test_add_dim.shape)
```

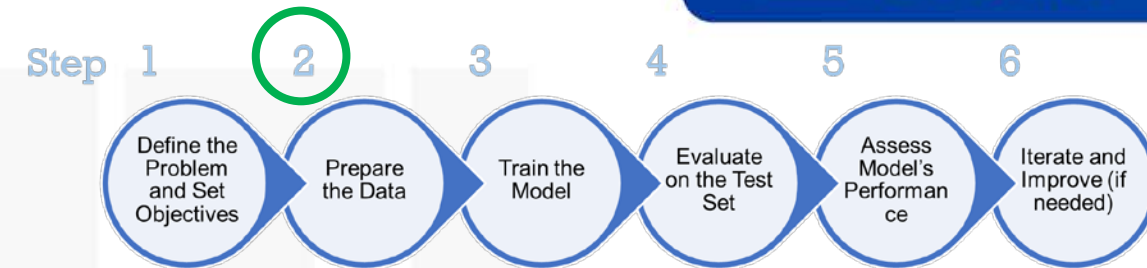


'''

Normalization: Normalization of image refers to keeping the pixel values in between 0 and 1. Normalizing helps stabilize and speed up the training process.

'''

```
X_train_add_dim_normalization = X_train_add_dim/255
X_test_add_dim_normalization = X_test_add_dim/255
```



Split train and validation

```
from sklearn.model_selection import train_test_split
```

```
X_train_1, X_val_1, y_train_1, y_val_1 =  
train_test_split(X_train_add_dim_normalization, y_train, test_size = 0.2,  
random_state=42)
```

random_state is a parameter in train_test_split that controls the random number generator used to shuffle the data
before splitting it. In other words, it ensures that the same randomization is used each time you run the code,
resulting in the same splits of the data. default=None.
random_state simply sets a seed to the random generator, so that your train-
test splits are always deterministic.
If you don't set a seed, it is different each time.

ลำดับ pipeline

Load data

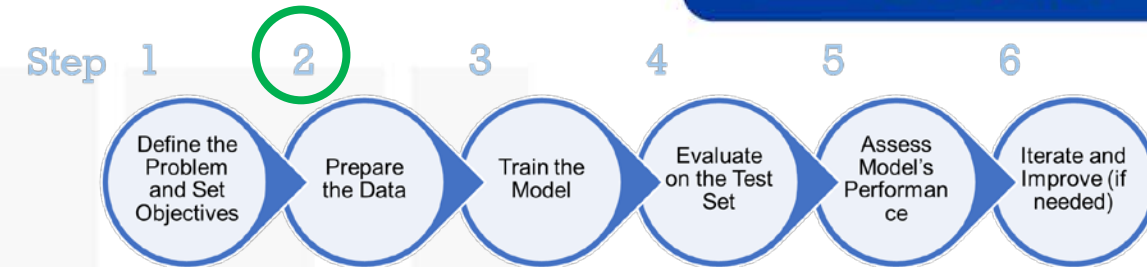
Add dimension (channel)

Normalize ($\div 255$)

Split Train / Validation

Train model

Test กับ Test set จริง



`X_train_1.shape, y_train_1.shape, X_val_1.shape, y_val_1.shape`

Step 1

2

3

4

5

6

Define the
Problem and Set
ObjectivesPrepare
the DataTrain the
ModelEvaluate
on the Test
SetAssess
Model's
PerformanceIterate and
Improve (if
needed)

```
model = keras.models.Sequential([  
    keras.layers.Input(shape=(28, 28, 1)),  
    keras.layers.Conv2D(filters = 16, activation='relu', kernel_size = 3, padding="same",  
                        name="conv1"),  
    keras.layers.MaxPooling2D(pool_size=(2,2)),  
    keras.layers.Activation("relu"),  
  
    keras.layers.Conv2D(32, 3, padding="same", name="conv2"),  
    keras.layers.MaxPooling2D(pool_size=(2,2)),  
    keras.layers.Activation("relu"),  
  
    keras.layers.Flatten(),  
    keras.layers.Dropout(.5),  
    keras.layers.Dense(15,activation='softmax', name="output_layer"),  
])
```

Define the
Problem and
Set
Objectives

Prepare
the Data

Train the
Model

Evaluate
on the Test
Set

Assess
Model's
Performan
ce

Iterate and
Improve (if
needed)

model.summary()

Layer (type)	Output Shape	Param #
conv1 (Conv2D)	(None, 28, 28, 16)	160
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 16)	0
activation_4 (Activation)	(None, 14, 14, 16)	0
conv2 (Conv2D)	(None, 14, 14, 32)	4,640
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 32)	0
activation_5 (Activation)	(None, 7, 7, 32)	0
flatten_2 (Flatten)	(None, 1568)	0
dropout_2 (Dropout)	(None, 1568)	0
output_layer (Dense)	(None, 15)	23,535

Total params: 28,335 (110.68 KB)

Trainable params: 28,335 (110.68 KB)

Non-trainable params: 0 (0.00 B)

Step 1

2

3

4

5

6

Define the
Problem
and Set
ObjectivesPrepare
the DataTrain the
ModelEvaluate
on the Test
SetAssess
Model's
PerformanceIterate and
Improve (if
needed)

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Conv2D, MaxPooling2D, Dropout, Flatten
def cnn_model(input_shape=(28, 28, 1)):
    model = Sequential()
    # Conv Block 1: 32 Filters, MaxPool.
    model.add(Conv2D(filters=32, kernel_size=3, padding='same', activation='relu', input_shape=input_shape))
    model.add(Conv2D(filters=32, kernel_size=3, padding='same', activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    # Conv Block 2: 64 Filters, MaxPool.
    model.add(Conv2D(filters=64, kernel_size=3, padding='same', activation='relu'))
    model.add(Conv2D(filters=64, kernel_size=3, padding='same', activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    # Conv Block 3: 64 Filters, model.add(Conv2D(filters=64, kernel_size=3, padding='same', activation='relu'))
    model.add(Conv2D(filters=64, kernel_size=3, padding='same', activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    # Flatten the convolutional features.
    model.add(Flatten())
    model.add(Dense(512, activation='relu'))
    model.add(Dense(10, activation='softmax'))
    return model
```

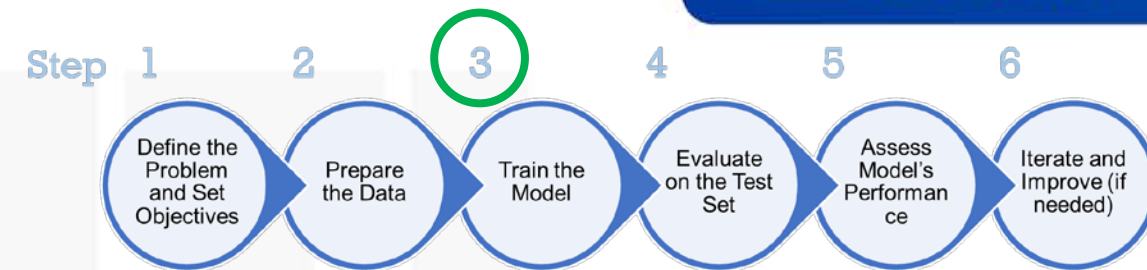
```
# Create the model.
model = cnn_model()
model.summary()
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 32)	320
conv2d_1 (Conv2D)	(None, 28, 28, 32)	9,248
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_2 (Conv2D)	(None, 14, 14, 64)	18,496
conv2d_3 (Conv2D)	(None, 14, 14, 64)	36,928
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 64)	0
conv2d_4 (Conv2D)	(None, 7, 7, 64)	36,928
conv2d_5 (Conv2D)	(None, 7, 7, 64)	36,928
max_pooling2d_4 (MaxPooling2D)	(None, 3, 3, 64)	0
flatten_1 (Flatten)	(None, 576)	0
dense (Dense)	(None, 512)	295,424
dense_1 (Dense)	(None, 10)	5,130

Total params: 439,402 (1.68 MB)

Trainable params: 439,402 (1.68 MB)

Non-trainable params: 0 (0.00 B)



```
model.compile(optimizer= 'adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])
```

```
EPOCHS = 10
```

```
history = model.fit(X_train_1, y_train_1, batch_size=512,  
                   epochs=EPOCHS, verbose = 1,  
                   validation_data = (X_val_1, y_val_1))
```

Step 1

2

3

4

5

6

Define the
Problem
and Set
ObjectivesPrepare
the DataTrain the
ModelEvaluate
on the Test
SetAssess
Model's
PerformanceIterate and
Improve (if
needed)KMITL
สถาบันเทคโนโลยี
พระจอมเกล้า
เจ้าพระยาKMITL
FIGHT

Epoch 1/10

94/94  **4s** 31ms/step - accuracy: 0.4424 - loss: 1.6480 - val_accuracy: 0.7791 - val_loss: 0.6094

Epoch 2/10

94/94  **2s** 21ms/step - accuracy: 0.7745 - loss: 0.6233 - val_accuracy: 0.8294 - val_loss: 0.4903

Epoch 3/10

94/94  **2s** 20ms/step - accuracy: 0.8094 - loss: 0.5216 - val_accuracy: 0.8479 - val_loss: 0.4336

Epoch 4/10

94/94  **2s** 21ms/step - accuracy: 0.8290 - loss: 0.4751 - val_accuracy: 0.8547 - val_loss: 0.4103

Epoch 5/10

94/94  **2s** 18ms/step - accuracy: 0.8376 - loss: 0.4442 - val_accuracy: 0.8632 - val_loss: 0.3878

Epoch 6/10

94/94  **3s** 29ms/step - accuracy: 0.8475 - loss: 0.4252 - val_accuracy: 0.8689 - val_loss: 0.3763

Epoch 7/10

94/94  **3s** 32ms/step - accuracy: 0.8550 - loss: 0.4121 - val_accuracy: 0.8724 - val_loss: 0.3658

Epoch 8/10

94/94  **3s** 36ms/step - accuracy: 0.8599 - loss: 0.3893 - val_accuracy: 0.8773 - val_loss: 0.3510

Epoch 9/10

94/94  **4s** 37ms/step - accuracy: 0.8602 - loss: 0.3916 - val_accuracy: 0.8808 - val_loss: 0.3402

Epoch 10/10

94/94  **4s** 39ms/step - accuracy: 0.8641 - loss: 0.3750 - val_accuracy: 0.8813 - val_loss: 0.3321

<keras.src.callbacks.history.History at 0x1566b206d90>

Define the
Problem
and Set
ObjectivesPrepare
the DataTrain the
ModelEvaluate
on the Test
SetAssess
Model's
Performan
ceIterate and
Improve (if
needed)

#Plot the Training Results

```
from matplotlib.ticker import (MultipleLocator, FormatStrFormatter)
```

```
# Retrieve training results.
```

```
train_loss = history.history["loss"]
```

```
train_acc = history.history["accuracy"]
```

```
valid_loss = history.history["val_loss"]
```

```
valid_acc = history.history["val_accuracy"]
```

```
plot_results([ train_loss, valid_loss ],
```

```
    ylabel="Loss",
```

```
    ylim = [0.0, 5.0],
```

```
    metric_name=["Training Loss", "Validation Loss"],
```

```
    color=["g", "b"]));
```

```
plot_results([ train_acc, valid_acc ],  
             ylabel="Accuracy",  
             ylim = [0.0, 1.0],  
             metric_name=["Training Accuracy", "Validation  
Accuracy"],  
             color=["g", "b"])
```

Step 1

2

3

4

5

6

Define the
Problem
and Set
Objectives

Prepare
the Data

Train the
Model

Evaluate
on the Test
Set

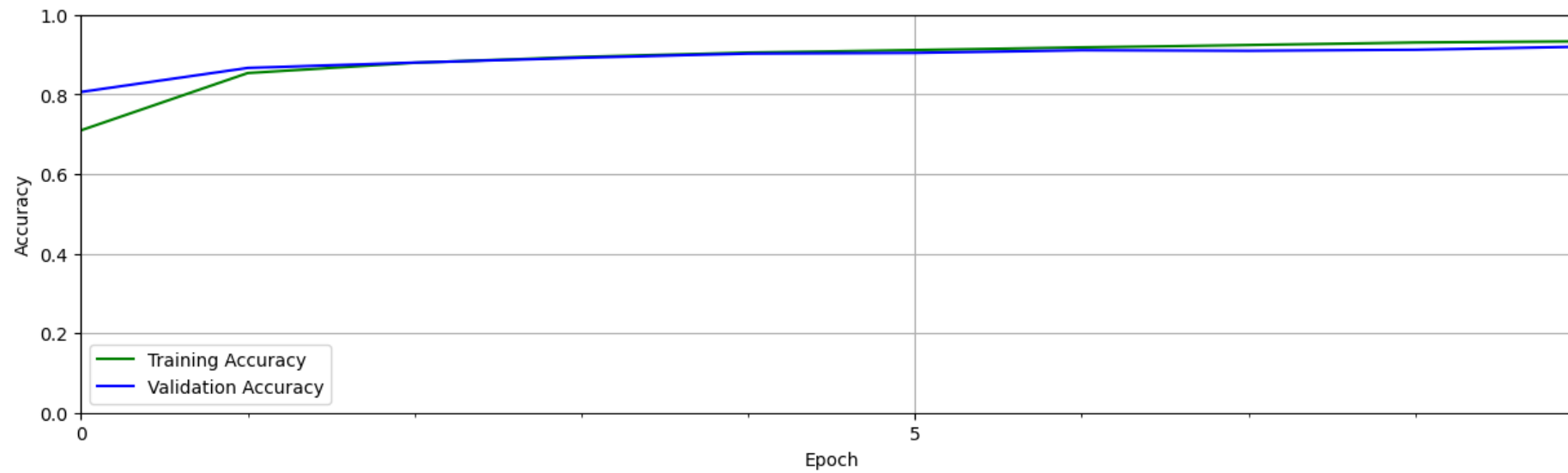
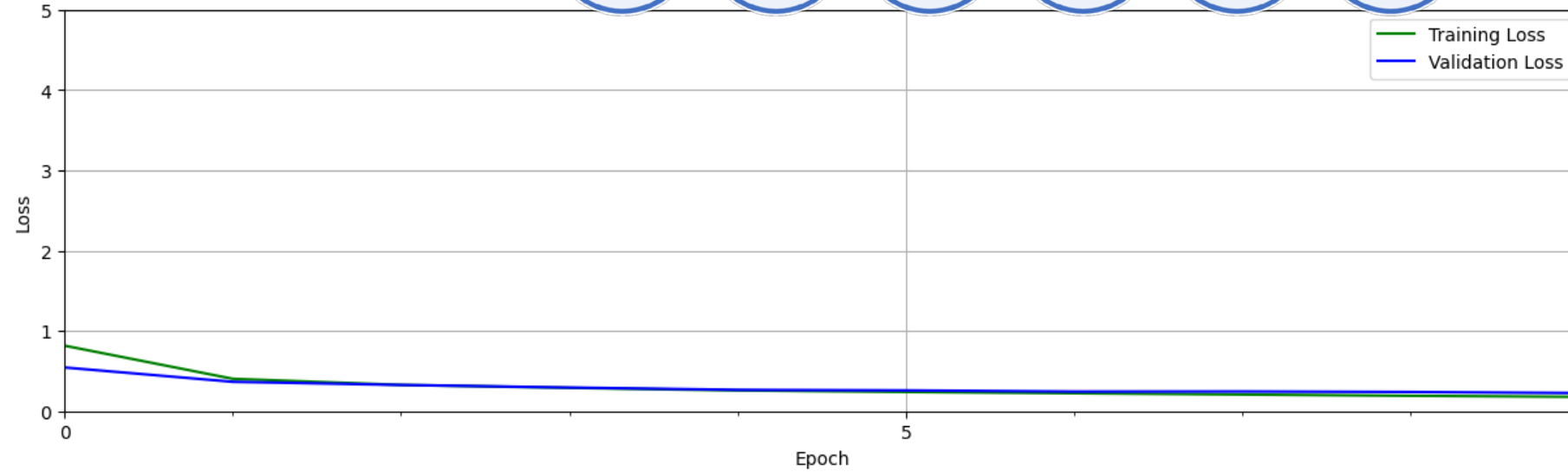
Assess
Model's
Performan
ce

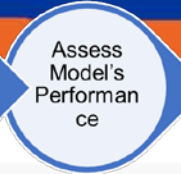
Iterate and
Improve (if
needed)



KMITL
สถาบันเทคโนโลยี
พระจอมเกล้า
เจ้าพระยา

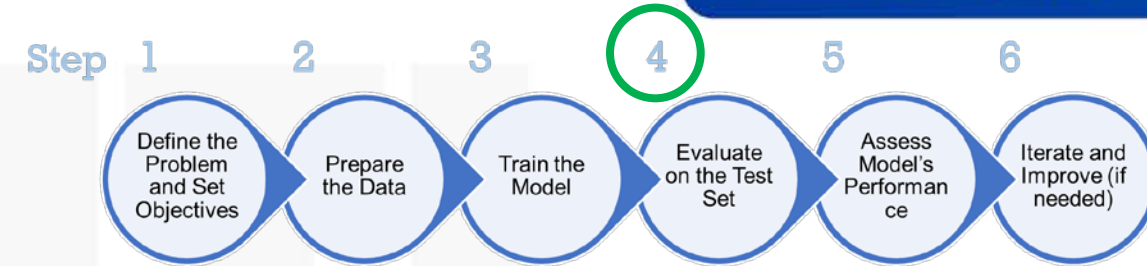
KMITL
FIGHT





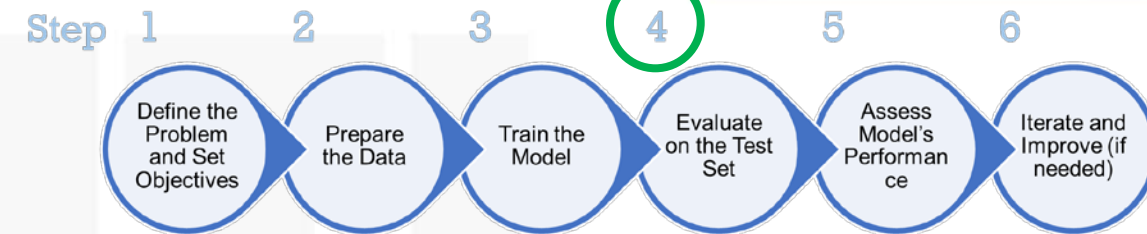
Using the save() method, the model will be saved to the file system in the 'SavedModel' format.

```
model.save('model_day3.h5')
```



```
from tensorflow.keras import models  
reloaded_model = models.load_model('model_day3.h5')
```

Evaluate the Model on the Test Dataset



```
test_loss, test_acc =  
reloaded_model.evaluate(X_test_add_dim_normalization, y_test)  
print(f"Test accuracy: {test_acc*100:.3f}")
```

313/313 ————— 3s 10ms/step - accuracy: 0.9123 - loss: 0.2438
Test accuracy: 91.360

```
# Generate predictions for the test dataset.
```

```
predictions = reloaded_model.predict(X_test_add_dim_normalization)
```

```
# For each sample image in the test dataset, select the class label with the highest probability.
```

```
predicted_labels = [np.argmax(i) for i in predictions]
```

```
# Let's see the confusion matrix
```

```
actual_labels = np.concatenate([y_test], axis=0)
```

```
# Compute the confusion matrix
```

```
cm = tf.math.confusion_matrix(actual_labels, predicted_labels)
```

```
# Plot the confusion matrix
```

```
plt.figure(figsize=(10, 8))
```

```
import seaborn as sn
```

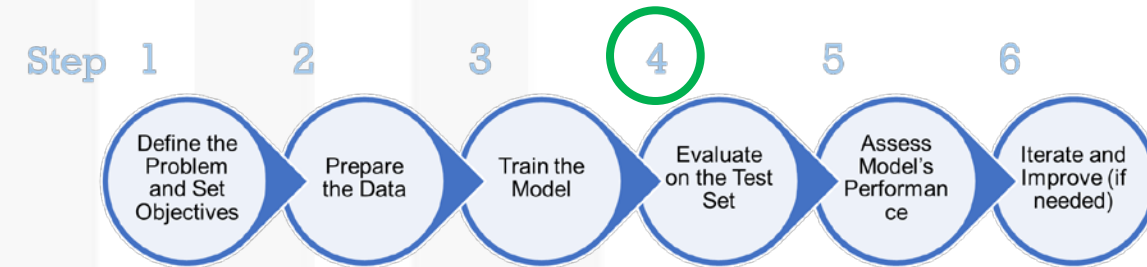
```
sn.heatmap(cm, annot=True, fmt='d', annot_kws={"size": 12})
```

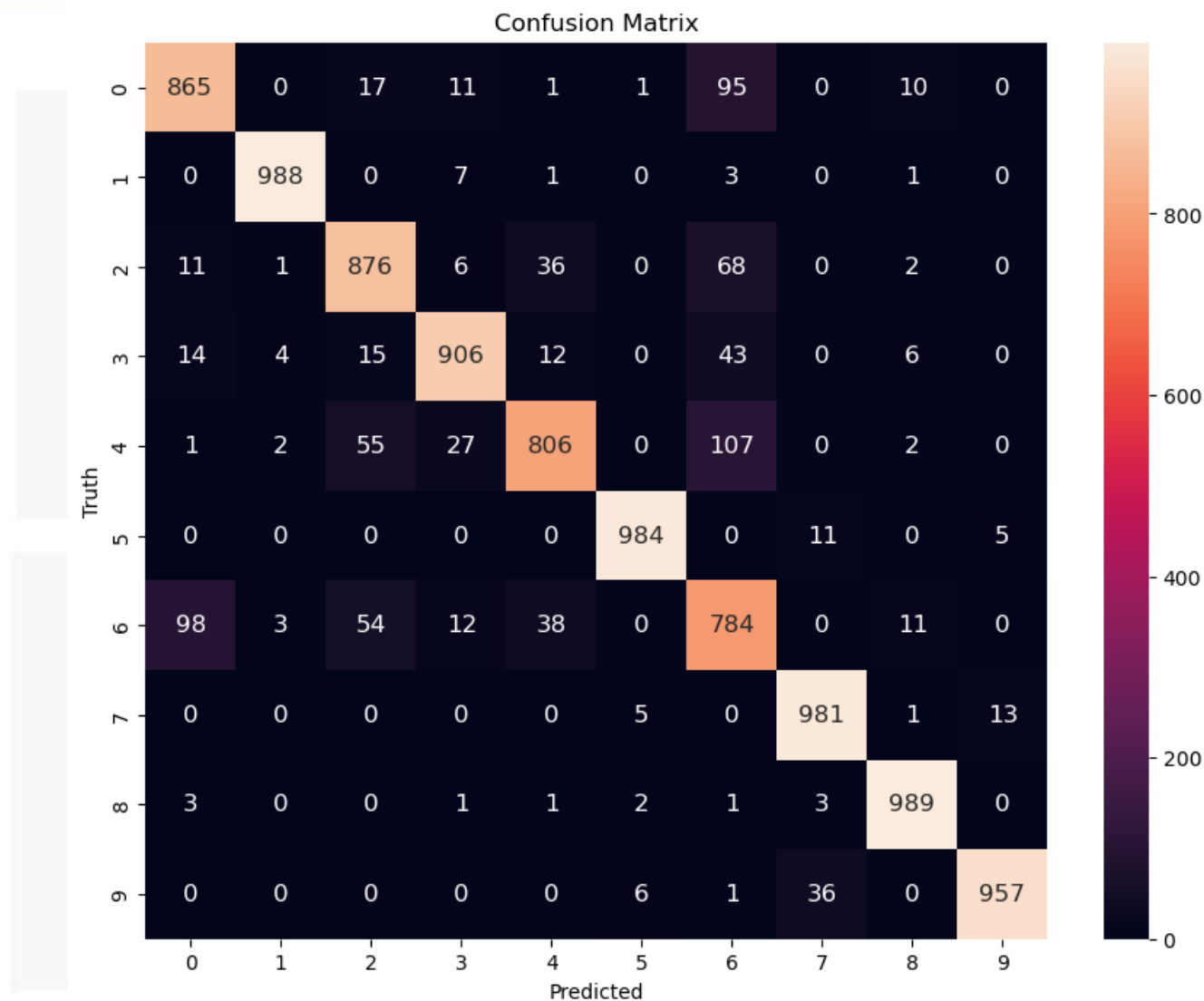
```
plt.title('Confusion Matrix')
```

```
plt.xlabel('Predicted')
```

```
plt.ylabel('Truth')
```

```
plt.show()
```





`class_labels = ['T-shirt/Top', 'Trouser', 'Pullover', 'Dress', 'coat', 'Sandal', 'shirt', 'Sneakers', 'Bag', 'Ankle Boot']`

Python Code (Predict regression model)

```

model = load_model(ii)
testY1 = model.predict(testX)
dfResult = pd.DataFrame()

dfResult["actual1"] = testY[0]
dfResult["actual2"] = testY[1]
dfResult["actual3"] = testY[2]
dfResult["predict1"] = testY1[:,0]
dfResult["predict2"] = testY1[:,1]
dfResult["predict3"] = testY1[:,2]
dfResult.columns = ["actual1","actual2","actual3","predict1","predict2","predict3"]

dfResult["diff1^2"] = dfResult["diff1"]*dfResult["diff1"]
dfResult["diff2^2"] = dfResult["diff2"]*dfResult["diff2"]
dfResult["diff3^2"] = dfResult["diff3"]*dfResult["diff3"]
dfResult["sqrdiffsum123^2"] = (dfResult["diff1^2"] + dfResult["diff2^2"] +dfResult["diff3^2"])

dfResult["name"] = str(i.rpartition('.')[0])
dfResult["sum rmse"] = dfResult["sqrdiffsum123^2"].sum()
f1 = (dfResult["sqrdiffsum123^2"].sum())/numtest
dfResult["avg rmse"] = f1

```

Python Code (Predict regression model) cont.

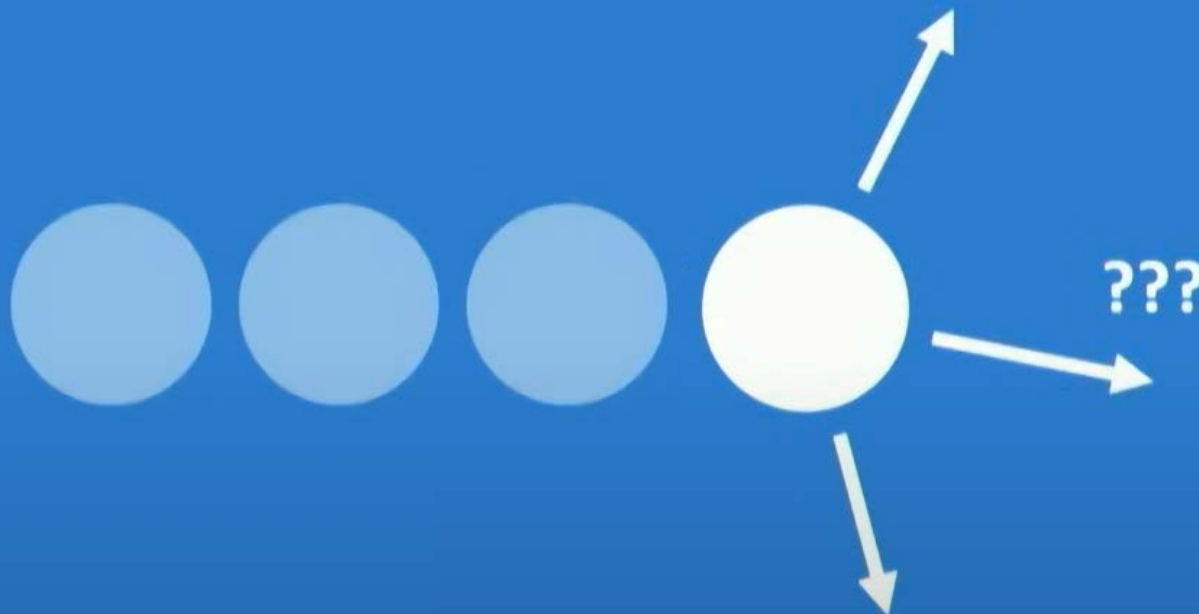
```
outputName = i.rpartition('.')[0]
dateTimestr = str(datetime.datetime.now().strftime("date%d%m%y_%Hh%Mm"))
dfResult.to_csv(DMnumofdataset + dateTimestr + outputName + ".csv")

print("name" ,i.rpartition('.')[0])
print("sumRmse" ,dfResult["sqrdiffsum123^2"].sum())
print("avgRmse" ,f1)
```

- name 128_128_128_16_adadeltadate111020_00h22m_yAll_model_2000data_200e
- sumRmse 226.9911710992821
- avgRmse 0.5254425256927826

- What is Deep learning
- Artificial neural network (ANN)
- Convolutional neural network (CNN)
- Recurrent Neural Network (RNN)
- Generative adversarial network (GAN)

Given an image of a ball,
can you predict where it will go next?



Sequence in the Wild



Audio

Sequence in the Wild

Character :

IT20



I

T

2

0

Word :

INTELLIGENT SYSTEM DEVELOPMENT



INTELLIGENT

SYSTEM

DEVELOPMENT

Text

Sequences in the Wild



Sequence Modeling Applications

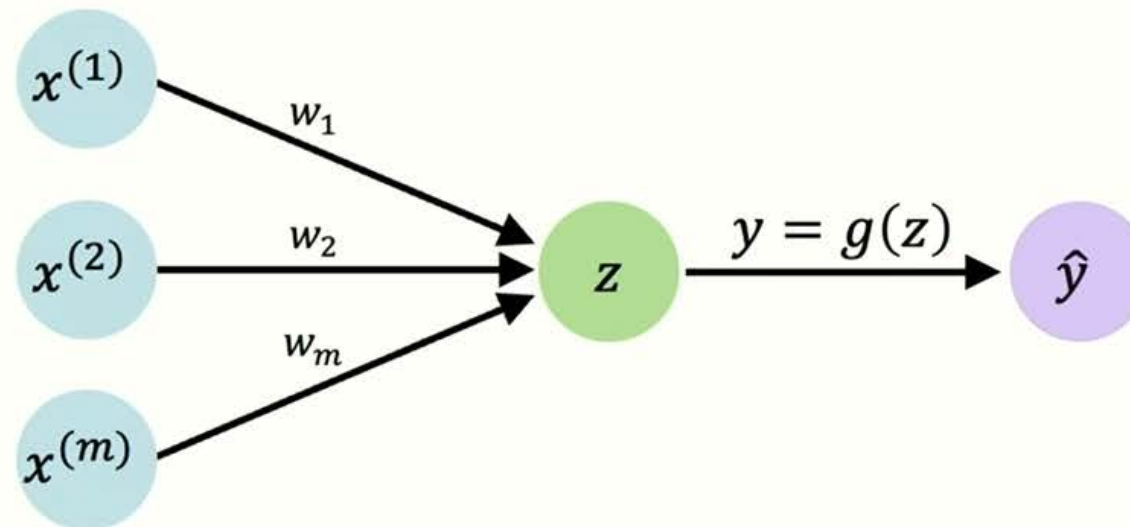


One to One
Binary Classification

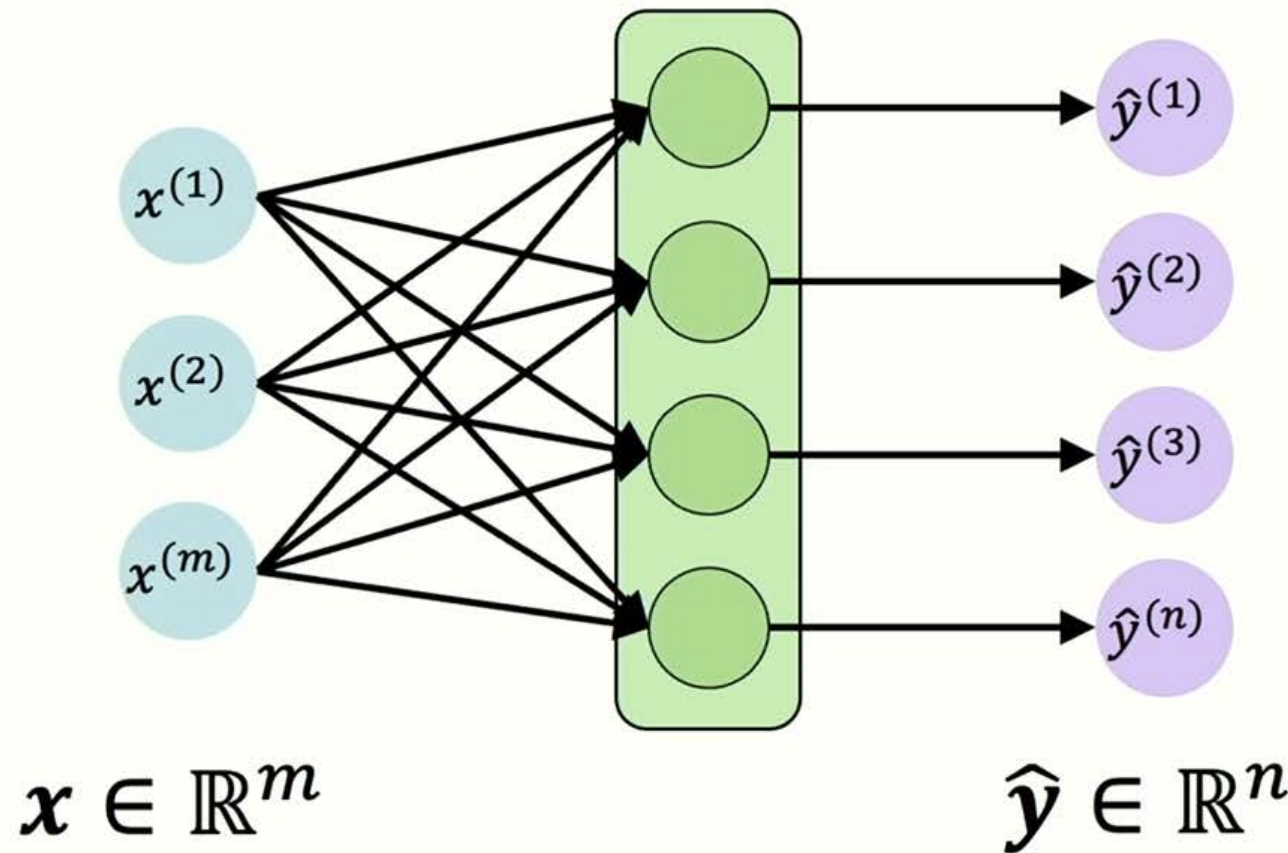


Neurons with Recurrence

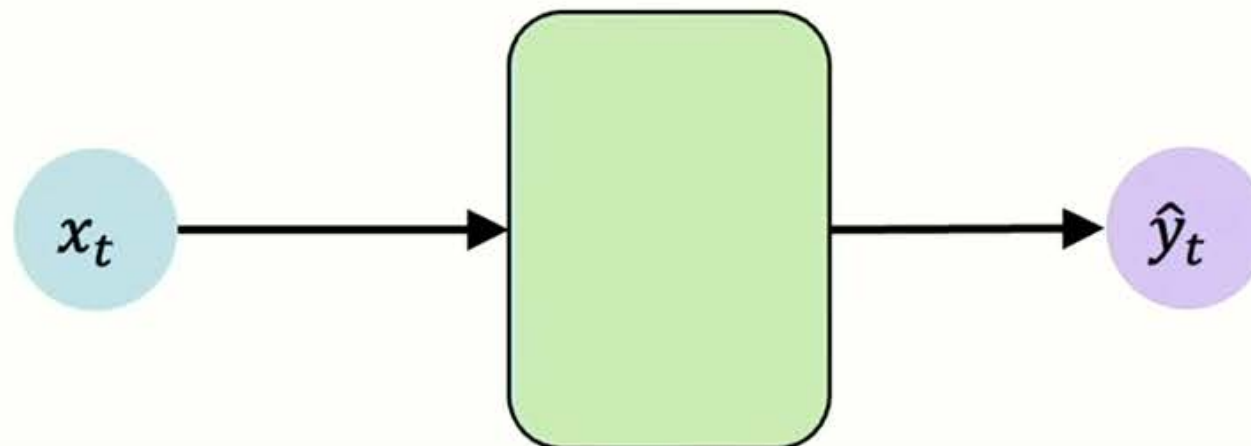
The Perceptron Revisited



Feed-Forward Networks Revisited



Feed-Forward Networks Revisited



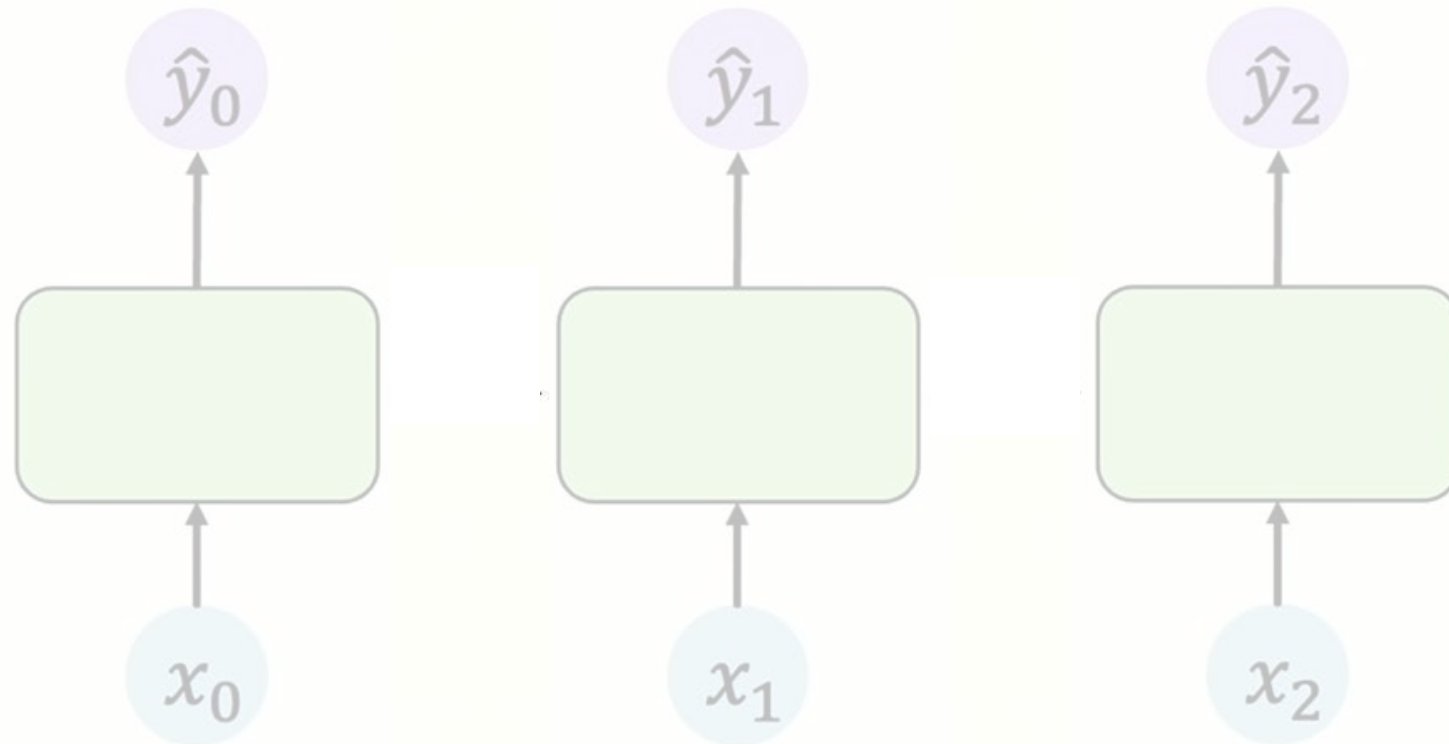
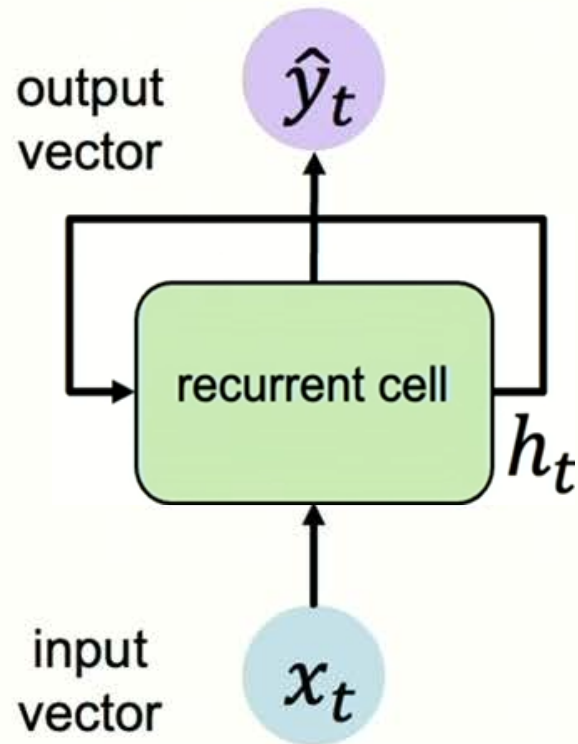
$$\mathbf{x}_t \in \mathbb{R}^m$$

$$\hat{\mathbf{y}}_t \in \mathbb{R}^n$$

Neurons with Recurrence

$$\hat{y}_t = f(\underbrace{x_t}_{\text{input}}, \underbrace{h_{t-1}}_{\text{past memory}})$$

output



Recurrent Neural Networks (RNNs)

Recurrent Neural Network (RNN)

Apply a **recurrence relation** at every time step to process a sequence:

$$h_t = f_W(x_t, h_{t-1})$$

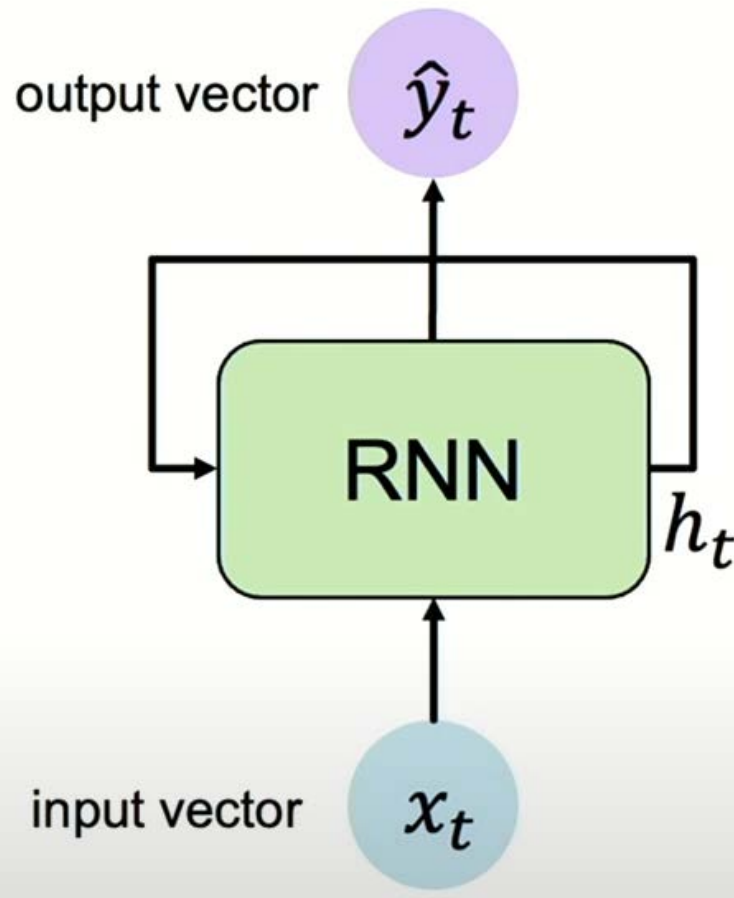
cell state

function
with weights
 W

input

old state

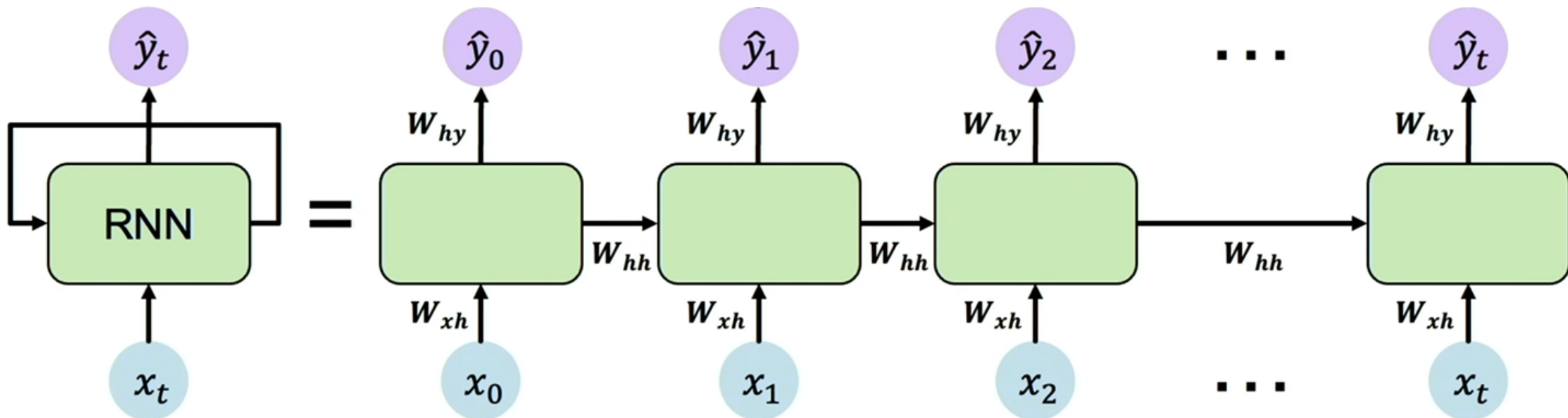
Note: the same function and set of parameters are used at every time step



RNNs have a **state**, h_t , that is updated **at each time step** as a sequence is processed

RNNs: Computational Graph Across Time

Re-use the **same weight matrices** at every time step



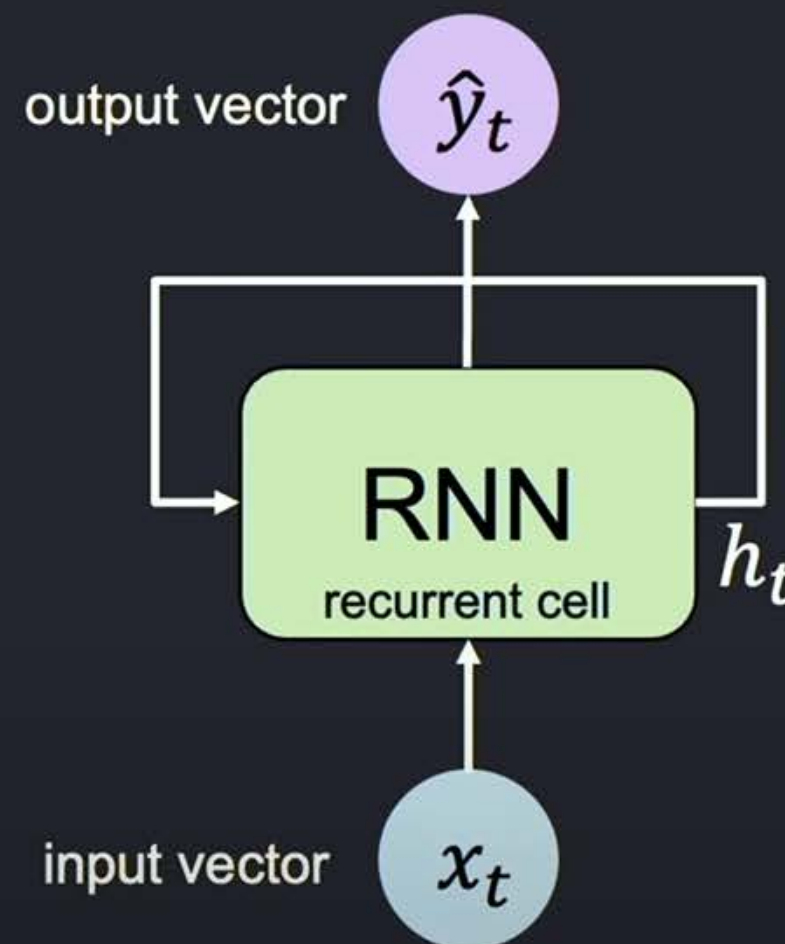
RNN Intuition

```
➔ my_rnn = RNN()
➔ hidden_state = [0, 0, 0, 0]

➔ sentence = ["I", "love", "recurrent", "neural"]

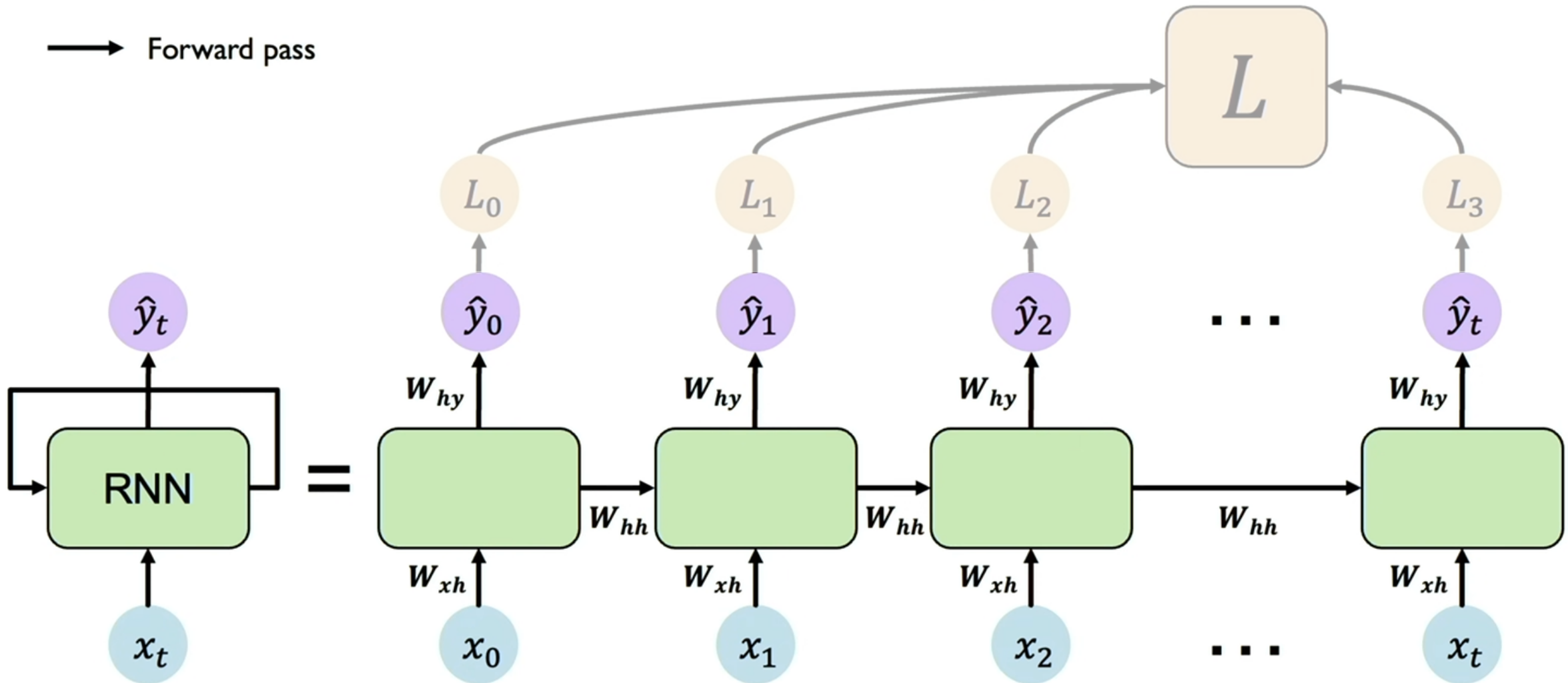
➔ for word in sentence:
➔     prediction, hidden_state = my_rnn(word, hidden_state)

➔ next_word_prediction = prediction
# >>> "networks!"
```



RNNs: Computational Graph Across Time

→ Forward pass



A Sequence Modeling Problem: Predict the Next Word

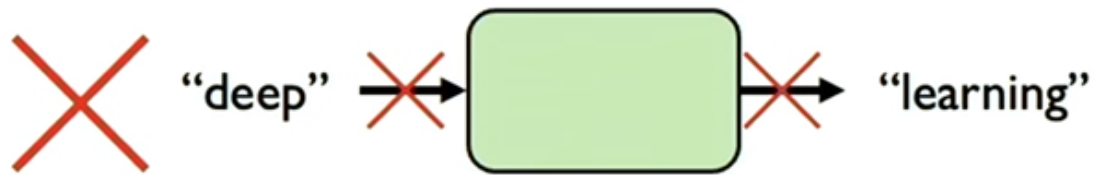
A Sequence Modeling Problem : Predict the next word

“This morning, I took my cat for a walk.”

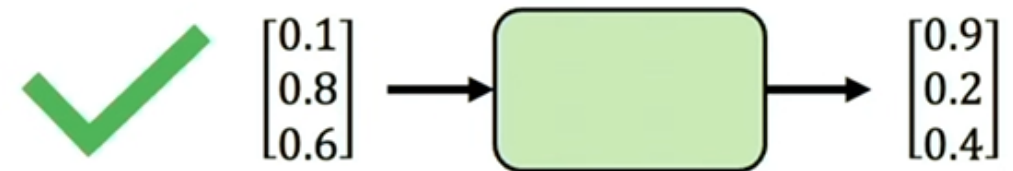
given these words

predict the
next word

Representing Language to a Neural Network

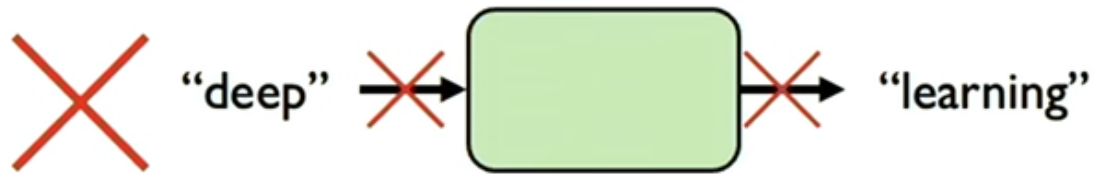


Neural networks cannot interpret words

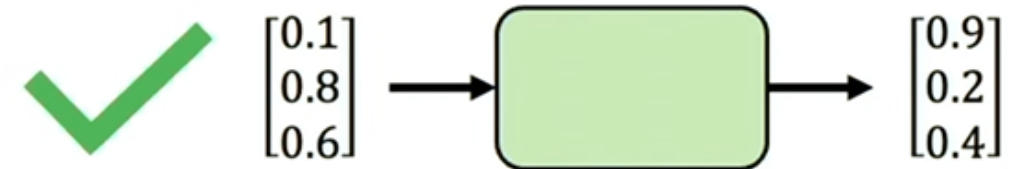


Neural networks require numerical inputs

Encoding Language for a Neural Network

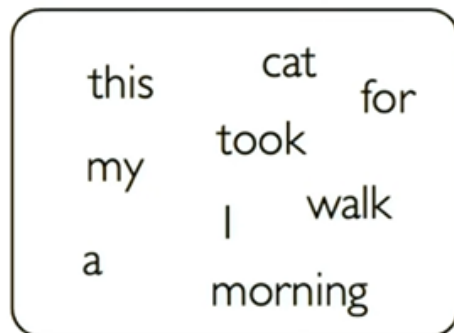


Neural networks cannot interpret words

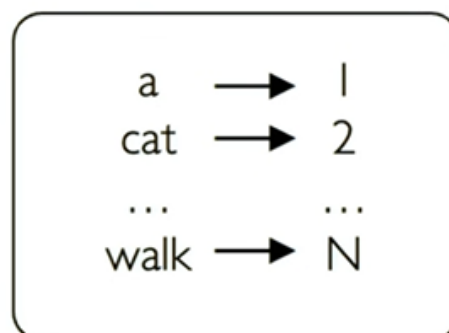


Neural networks require numerical inputs

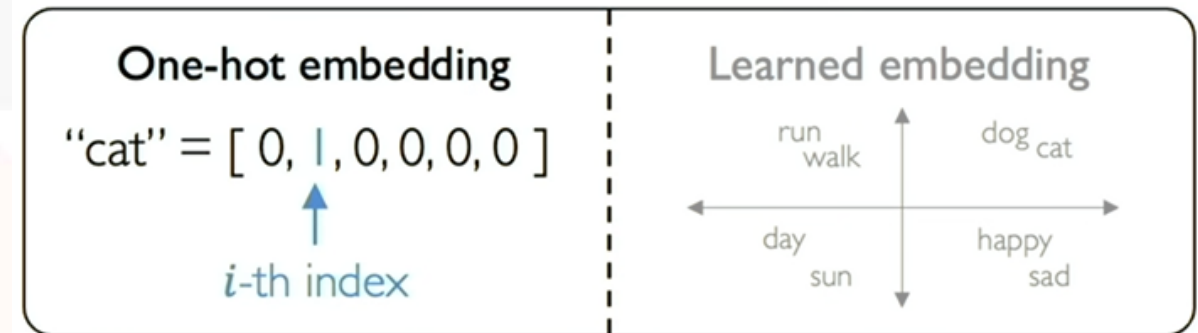
Embedding : transform indexes into a vector of fixed size.



1. Vocabulary:
Corpus of words



2. Indexing:
Word to index



3. Embedding:
Index to fixed-sized vector

Capture Differences in Sequence Order



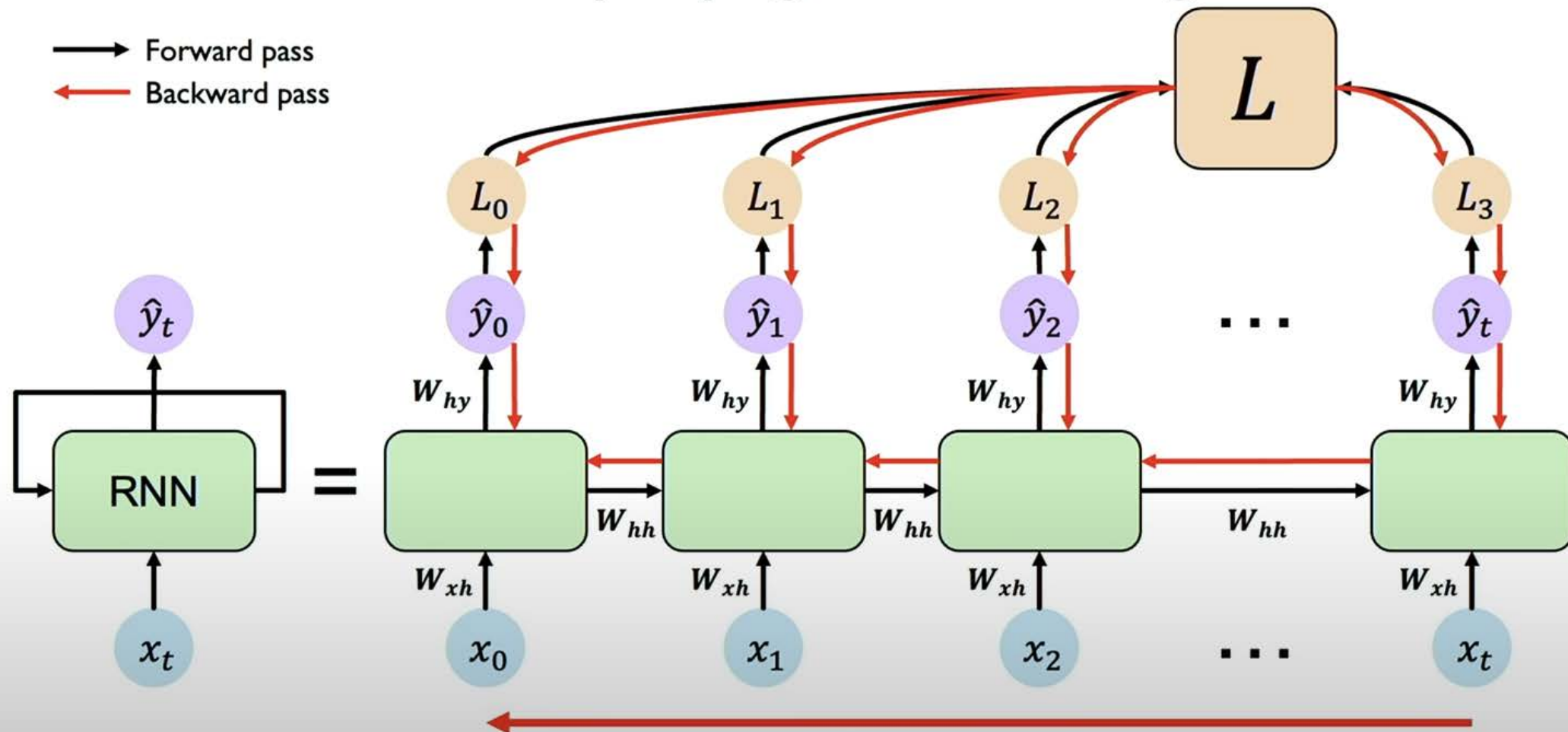
The food was good, not bad at all.

vs.

The food was bad, not good at all.



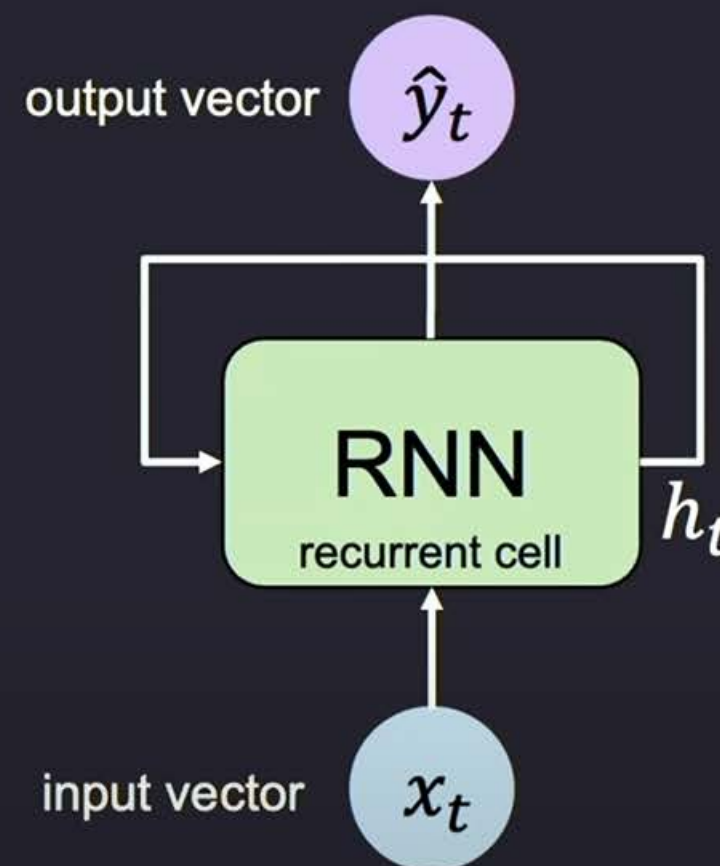
RNNs: Backpropagation Through Time





RNNs from Scratch

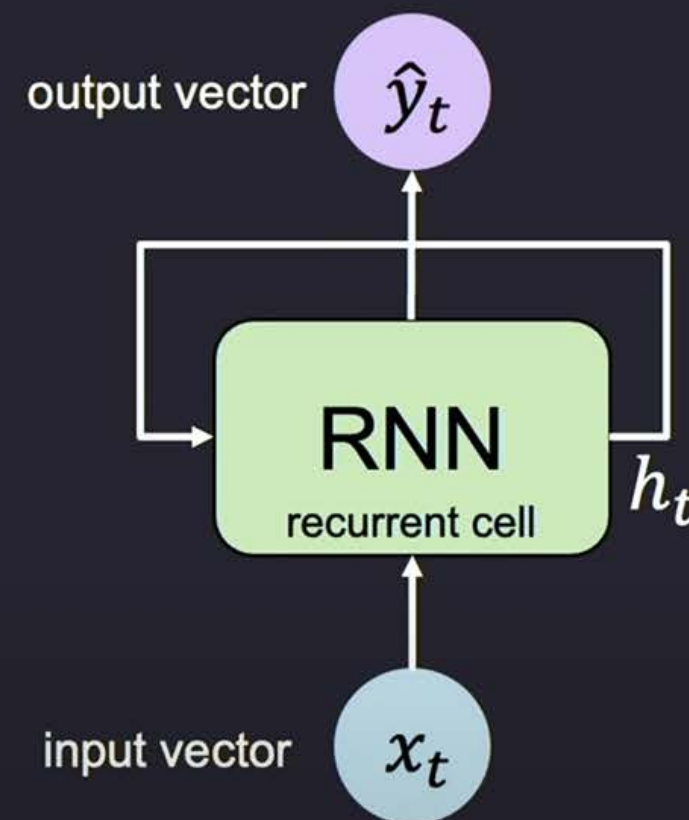
```
class MyRNNCell(tf.keras.layers.Layer):  
    def __init__(self, rnn_units, input_dim, output_dim):  
        super(MyRNNCell, self).__init__()  
  
        # Initialize weight matrices  
        self.W_xh = self.add_weight([rnn_units, input_dim])  
        self.W_hh = self.add_weight([rnn_units, rnn_units])  
        self.W_hy = self.add_weight([output_dim, rnn_units])  
  
        # Initialize hidden state to zeros  
        self.h = tf.zeros([rnn_units, 1])  
  
    def call(self, x):  
        # Update the hidden state  
        self.h = tf.math.tanh( self.W_hh * self.h + self.W_xh * x )  
  
        # Compute the output  
        output = self.W_hy * self.h  
  
        # Return the current output and hidden state  
        return output, self.h
```



RNN Implementation in TensorFlow



```
tf.keras.layers.SimpleRNN(rnn_units)
```



Lab — Week 4:

Format the Lab 3 output to match the ground truth (GT) using `_extraction.py`

Write `cli.py` to run `_extraction.py` end to end

Evaluate the results using the Lab 3 code provided

Push your work to Git as a group and merge it into the main branch; submit by 2:00 PM today